# Refactorización — Fichero Aleatorio

Actividad de refactorización con Eclipse — DAM

**Alumno: Oscar Alvarez Oliveira | 1º DAMDUAL**

## Apartados 1 al 4 — Extracción de variables y constantes

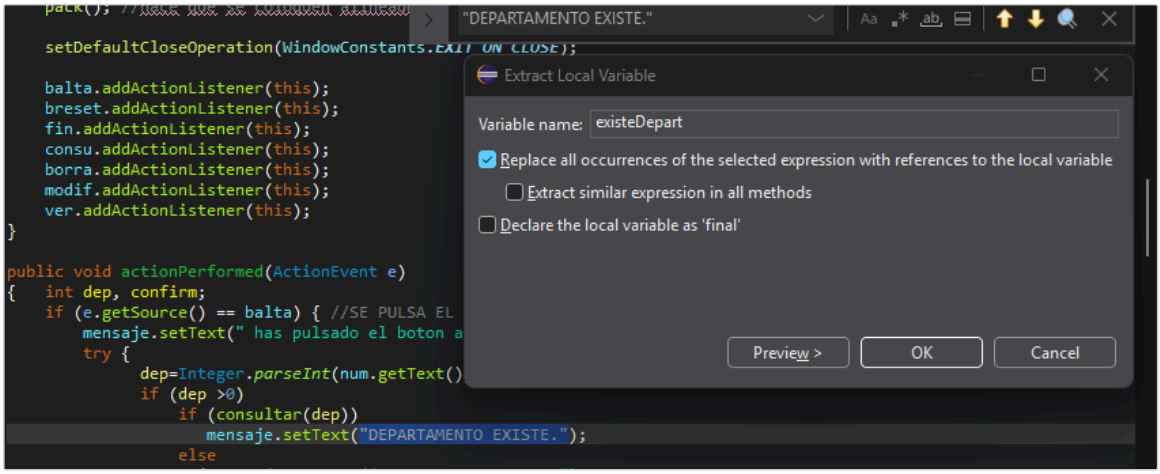
Extracción de cadenas literales a variables locales y posteriormente a atributos `static final` de clase.

```
23 JLabel mensaje=new JLabel(" ---------------------------- ");
24 JLabel titulo=new JLabel ("GESTIÓN DE DEPARTAMENTOS.");
25
26 JLabel lnum = new JLabel ("NUMERO DEPARTAMENTO:");
27 JLabel lnom = new JLabel ("NOMBRE:");
28 JLabel lloc = new JLabel ("LOCALIDAD:");
29
30 JButton balta= new JButton("Insertar Depar.t");
31 JButton consu= new JButton("Consultar Depart.");
32 JButton borra= new JButton("Borrar Depart.");
33 JButton breset=new JButton("Limpiar datos.");
34 JButton modif=new JButton("Modificar Departamento.");
35 JButton ver=new JButton("Ver por consola.");
36 JButton fin=new JButton("CERRAR");
37 Color c; //para poner colores
38 private static final String EXISTE_DEPART = "DEPARTAMENTO EXISTE";
39 private static final String DEPAR_ERROR = "DEPARTAMENTO ERRÓNEO.";
40  // WHITE,LIGHTGRAY,GRAY,DARKGRAY,BLUE,BLACK,RED,MAGENTA,PINK,ORANGE,CYAN,GREEN,YELLOW
41
42 public VentanaDepart(JFrame f )
43 {
44     setTitle("GESTIÓN DE DEPARTAMENTOS.");
45
```

*Resultado final: EXISTE_DEPART y DEPAR_ERROR declarados como private static final en la clase*

### Apartado 1 — Variable local `existeDepart`

Cadena `"DEPARTAMENTO EXISTE."` → *Refactor* → *Extract Local Variable* → `existeDepart`.



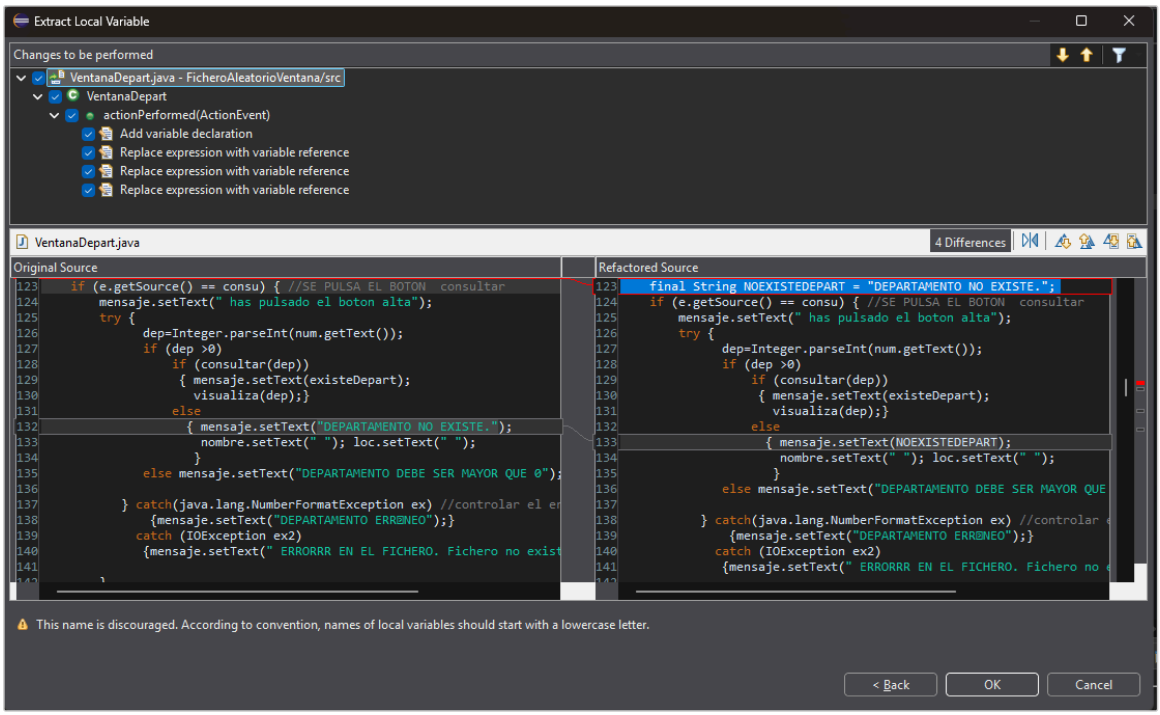

*Diálogo Extract Local Variable para existeDepart*

```
98     String existeDepart = "DEPARTAMENTO EXISTE";
```

*Variable local existeDepart en el código*

### Apartado 2 — Constante local `NOEXISTEDEPART`

Cadena `"DEPARTAMENTO NO EXISTE."` → Extract Local Variable marcando *Declare final*.



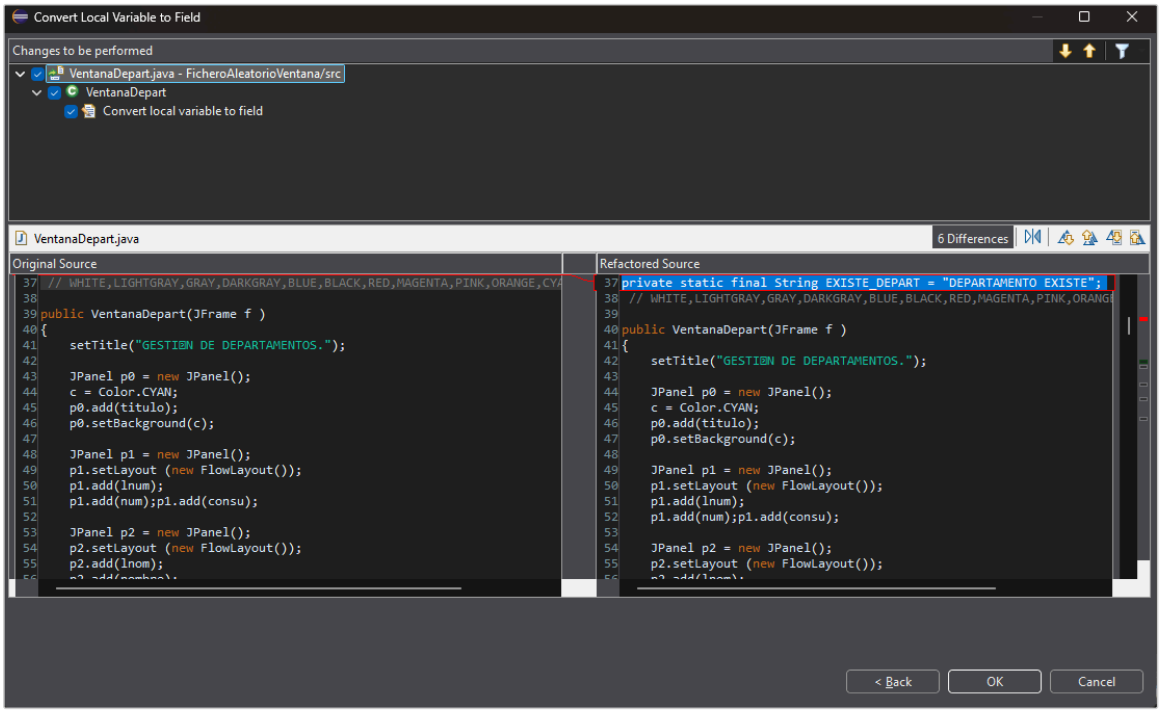

*Extract Local Variable con Declare final marcado*

```
final String NOEXISTEDEPART = "DEPARTAMENTO NO EXISTE";
```

*Constante NOEXISTEDEPART declarada como final*

### Apartado 3 — Atributo `EXISTE_DEPART`

*Refactor → Convert Local Variable to Field* con `static` y `final`. Renombrada a `EXISTE_DEPART` según convención Java.



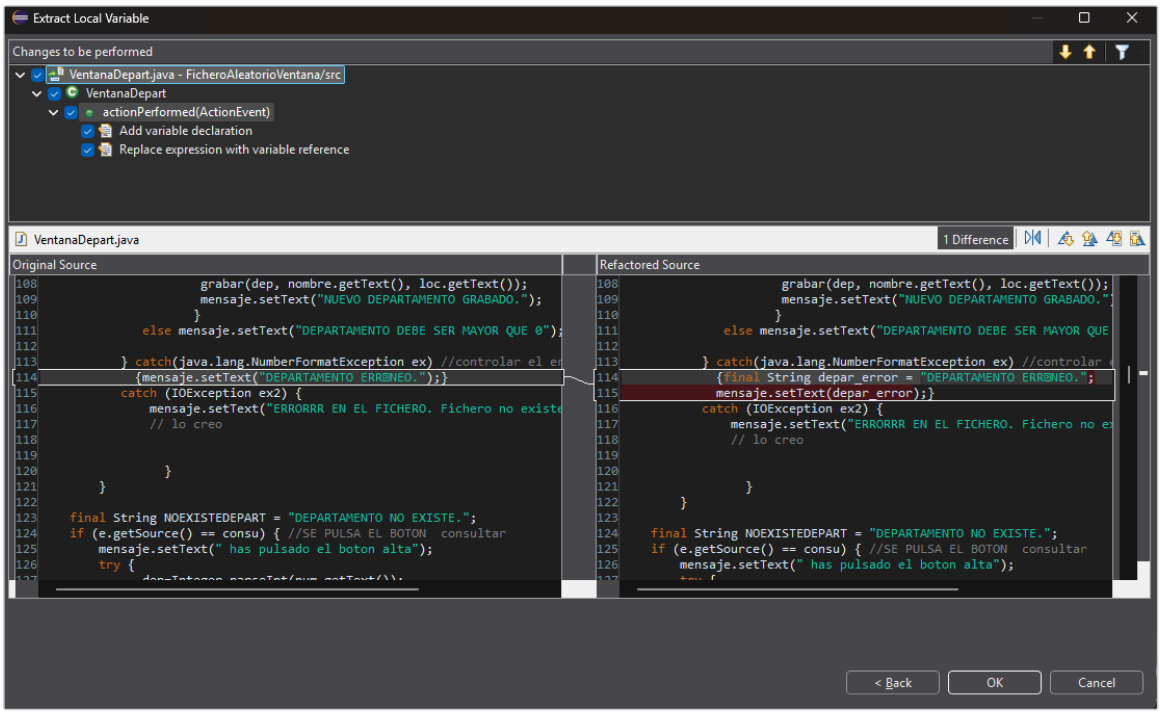

*Convert Local Variable to Field*

```
private static final String EXISTE_DEPART = "DEPARTAMENTO EXISTE";
```

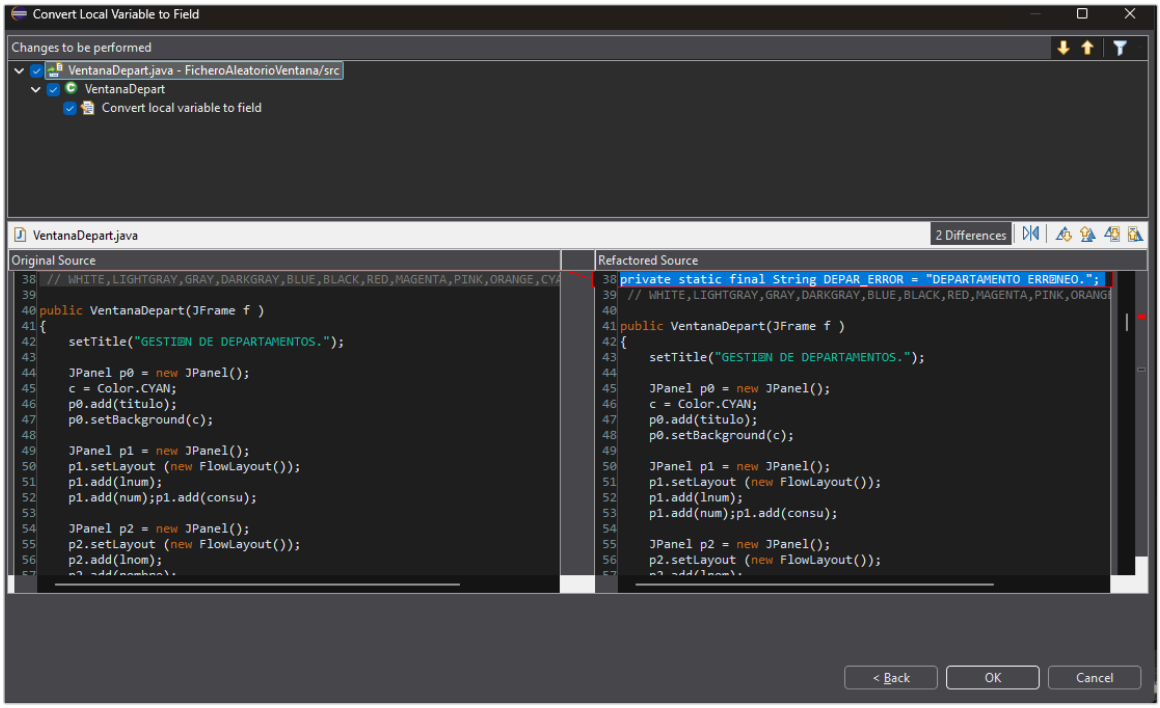
*EXISTE_DEPART como private static final*

### Apartado 4 — Atributo `DEPAR_ERROR`

Cadena `"DEPARTAMENTO ERRÓNEO."` → variable local `depar_error` → atributo `private static final DEPAR_ERROR`.




*Extracción de depar_error*



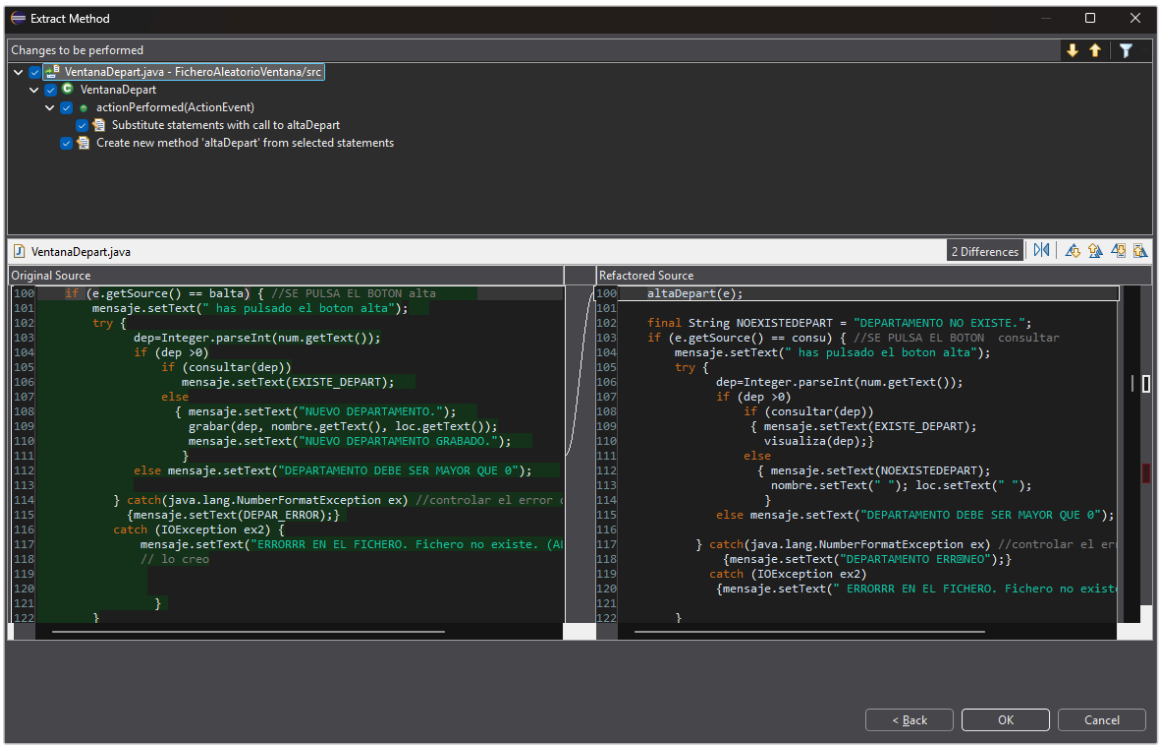

*Conversión a DEPAR_ERROR static final*

```
private static final String DEPAR_ERROR = "DEPARTAMENTO ERR□NEO.";
```

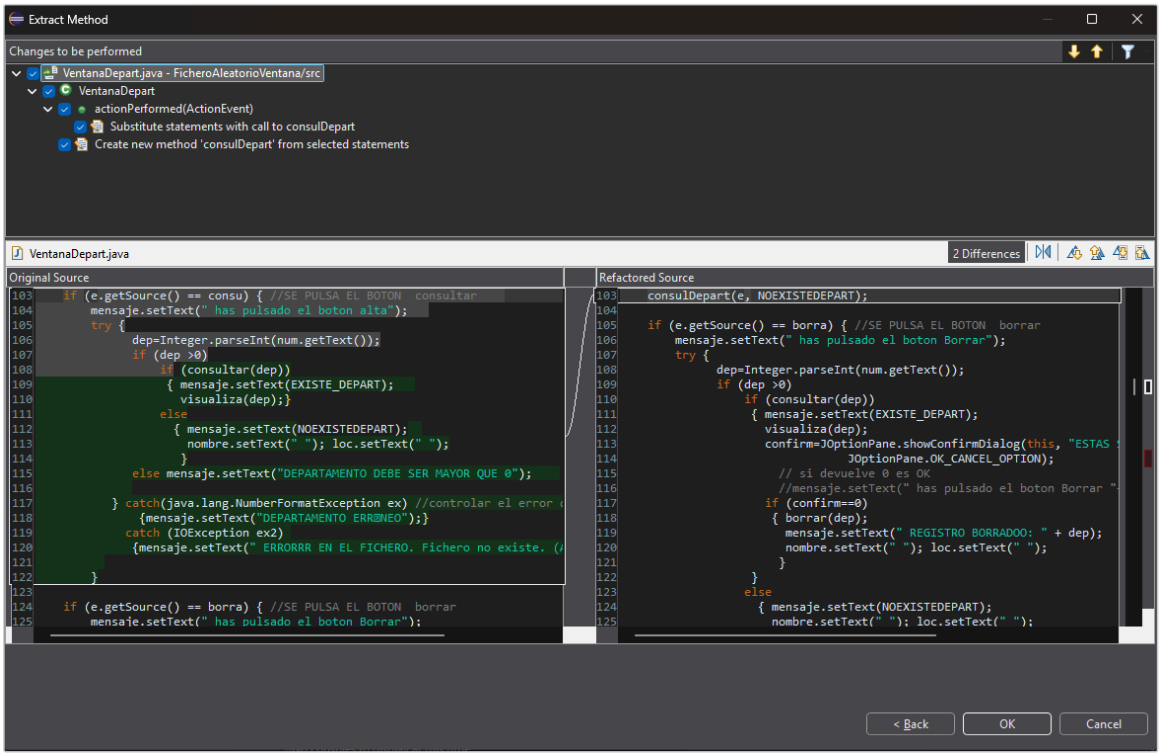
*DEPAR_ERROR como private static final*

## Apartado 5 — Extracción de métodos de `actionPerformed`

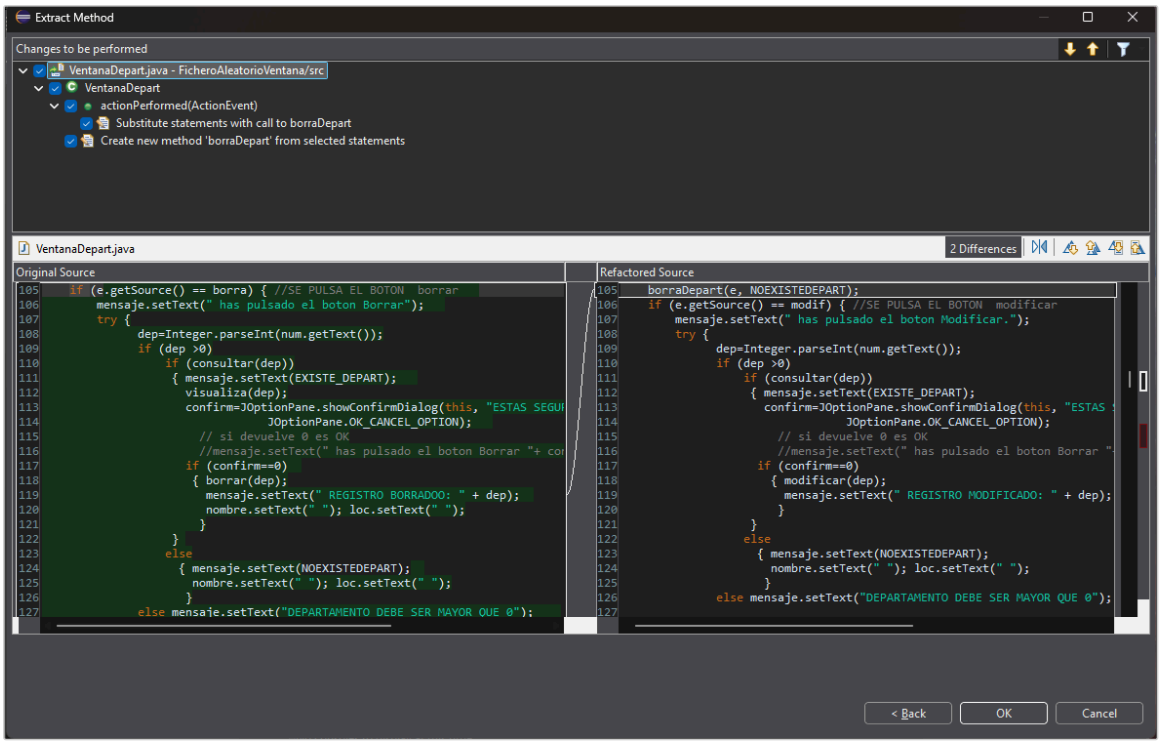
Cuatro bloques de lógica extraídos como métodos independientes con *Refactor → Extract Method*.



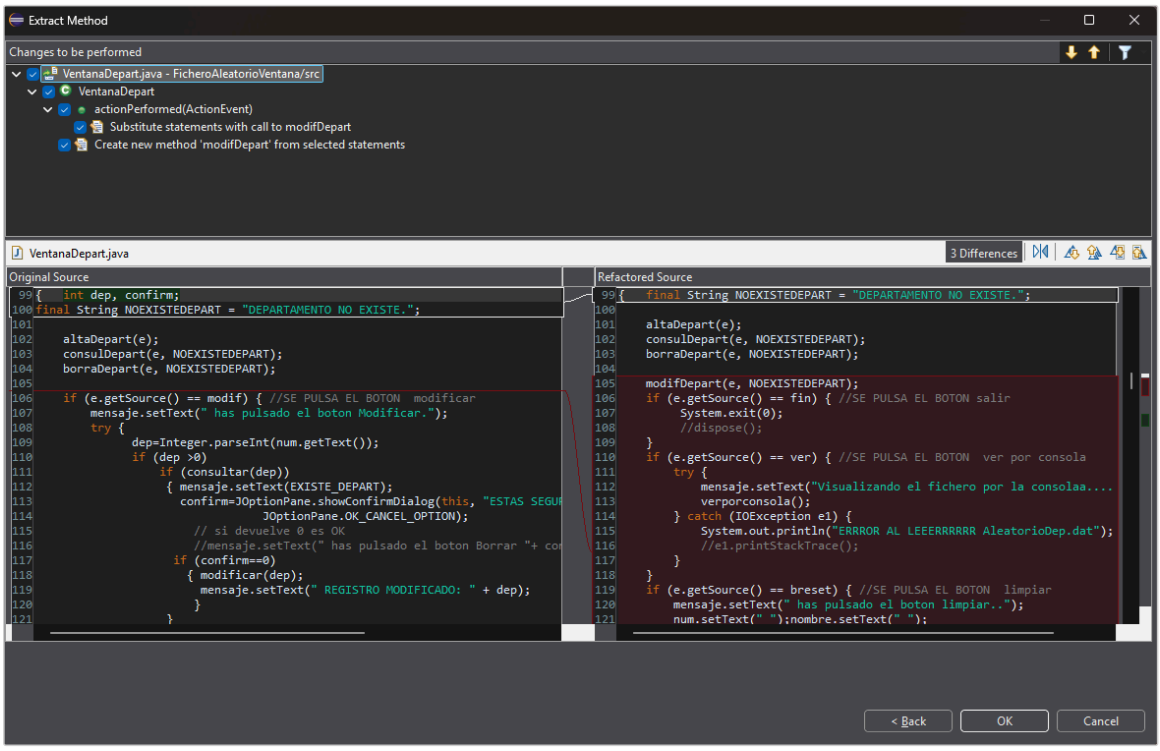

*Extracción de altaDepart*




*Extracción de consulDepart*




*Extracción de borraDepart*




*Extracción de modifDepart*

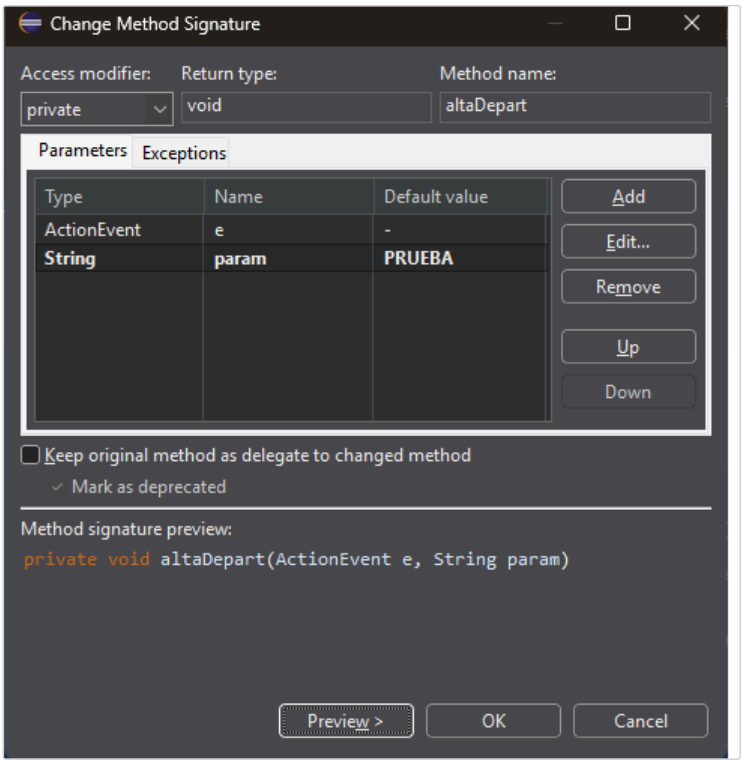
### Resultado final de `actionPerformed`

```java
public void actionPerformed(ActionEvent e) {

final String NOEXISTEDEPART = "DEPARTAMENTO NO EXISTE.";

    altaDepart(e);
    consulDepart(e, NOEXISTEDEPART);
    borraDepart(e, NOEXISTEDEPART);
    modifDepart(e, NOEXISTEDEPART);
```

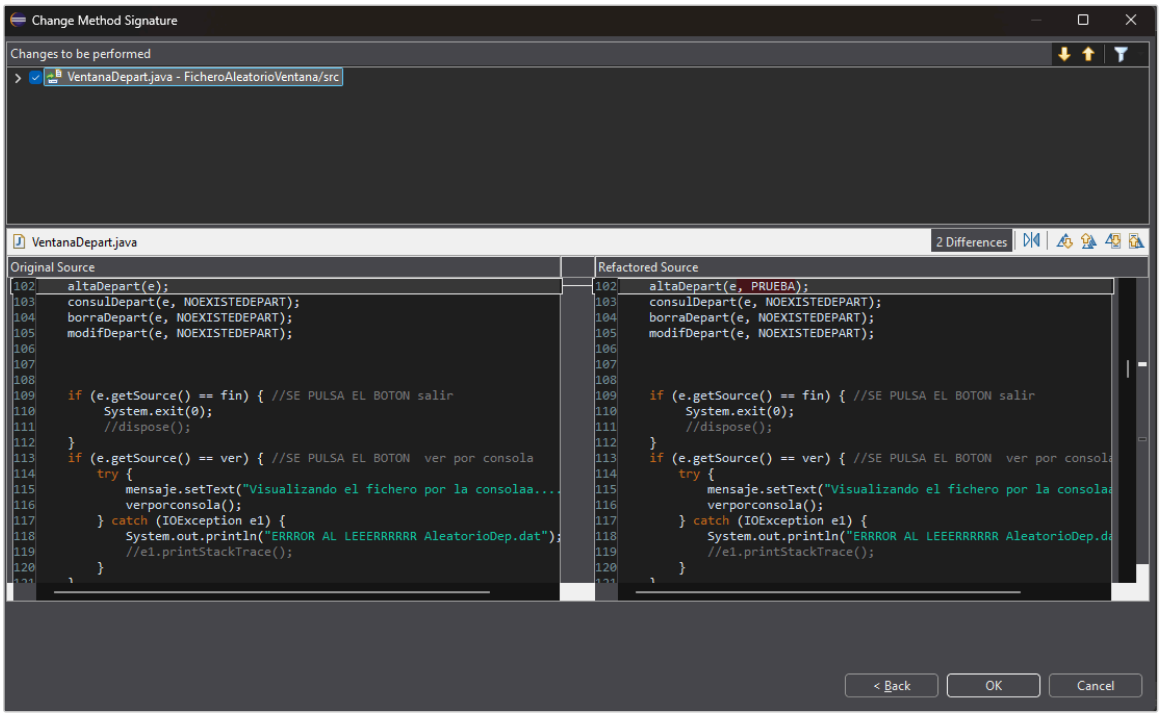
*actionPerformed con las cuatro llamadas extraídas*

## Apartado 6 — Cambio de firma de `altaDepart`

*Refactor → Change Method Signature*: se añadió `String param` con valor por defecto `"PRUEBA"` y se cambió el retorno a `int`. Se añadió `return 0;` para corregir el error de compilación.




*Change Method Signature: nuevo parámetro y retorno int*




*Preview: altaDepart(e, PRUEBA)*

```java
private int altaDepart(ActionEvent e, String param) {
    int dep;
    if (e.getSource() == balta) { //SE PULSA EL BOTON alta
        mensaje.setText(" has pulsado el boton alta");
        try {
              dep=Integer.parseInt(num.getText());
              if (dep >0)
                  if (consultar(dep))
                     mensaje.setText(EXISTE_DEPART);
                  else
                    { mensaje.setText("NUEVO DEPARTAMENTO.");
                      grabar(dep, nombre.getText(), loc.getText());
                      mensaje.setText("NUEVO DEPARTAMENTO GRABADO.");
                     }
              else mensaje.setText("DEPARTAMENTO DEBE SER MAYOR QUE 0");

           } catch(java.lang.NumberFormatException ex) //controlar el error del Integer.parseInt
             {mensaje.setText(DEPAR_ERROR);}
              catch (IOException ex2) {
                  mensaje.setText("ERRORRR EN EL FICHERO. Fichero no existe. (ALTA)");
                  // lo creo


                }
        }
            return 0;
}
```

*Método altaDepart con return 0 al final*

## Apartado 7 — Clase anidada `ClaseAnidada`

Clase `public ClaseAnidada` creada dentro de `VentanaDepart`. Al ser clase interna puede ser `public` sin conflicto.

```java
public class VentanaDepart extends JFrame implements ActionListener  {


    // Inicio clase anidada
    public class ClaseAnidada {
        void entrada() {
            System.out.println("Método de entrada. ");
        }

        String salida(int d) {
            System.out.println("Salida. ");
            return ("Salida el " + d);
        }
    }
    // fin clase anidada




private static final long serialVersionUID = 1L;
```

*ClaseAnidada anidada dentro de VentanaDepart*

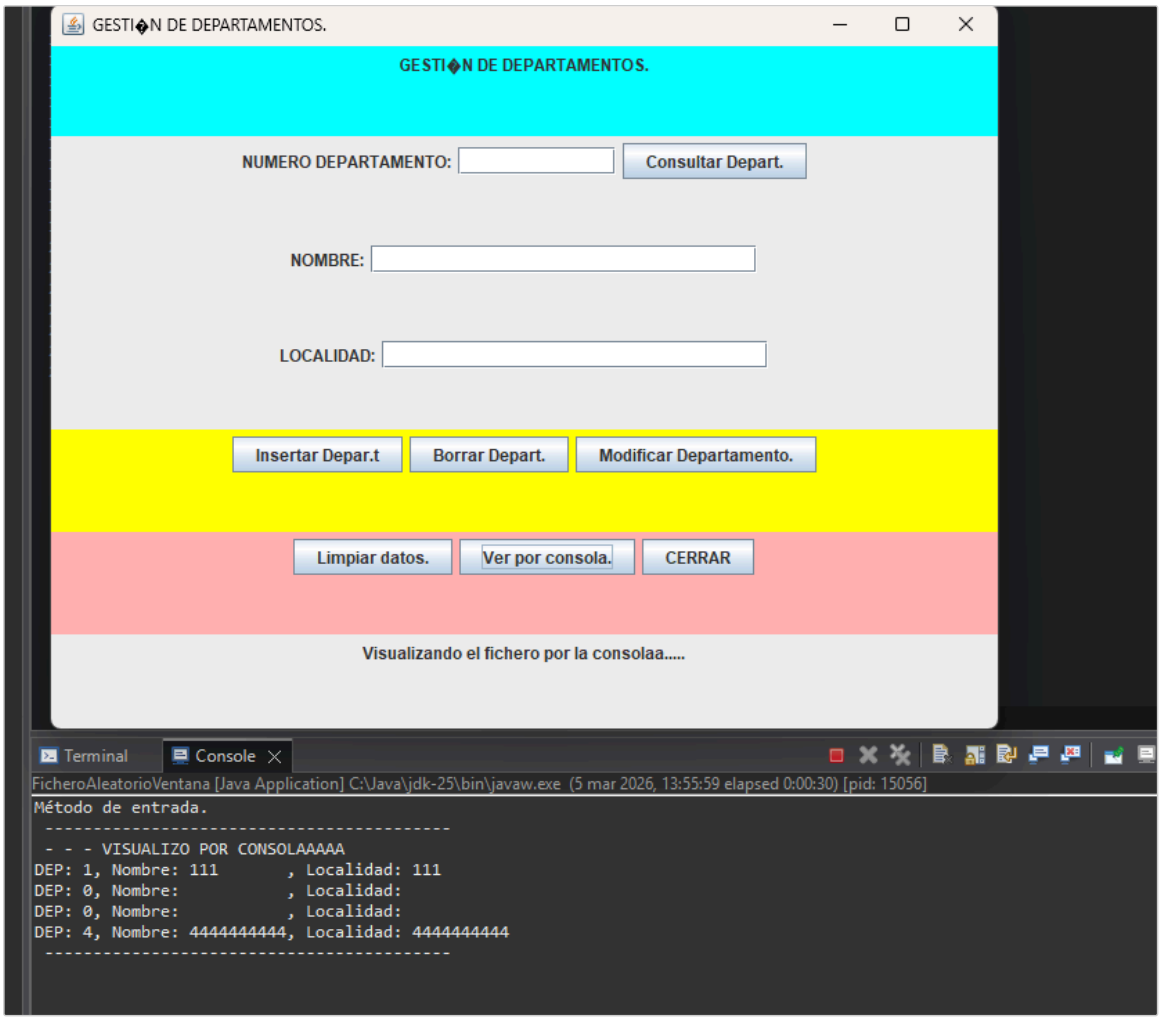
## Apartado 8 — Uso y conversión de la clase anidada

### a) Instanciación y ejecución

Código añadido en `verporconsola` para instanciar `ClaseAnidada` y llamar a `entrada()`. Verificado por consola.

```java
public  void verporconsola() throws IOException {

  String  nom="",loc=""; int dep=0; long pos;
  File fichero = new File("AleatorioDep.dat");
  RandomAccessFile file = new RandomAccessFile(fichero, "r");
  char cad[] = new char[10], aux;

  VentanaDepart fich = new VentanaDepart(null);
  ClaseAnidada ej = fich.new ClaseAnidada();
  ej.entrada();

  if (file.length()>0 ){
    pos=0;  //para situarnos al principio
    System.out.println(" ----------------------------------------");
    System.out.println(" - - - VISUALIZO POR CONSOLAAAAA ");
    for(;;){  //recorro el fichero, visualiza tambien las posiciones vacias
          file.seek(pos);
          dep=file.readInt();   // obtengo el dep
```

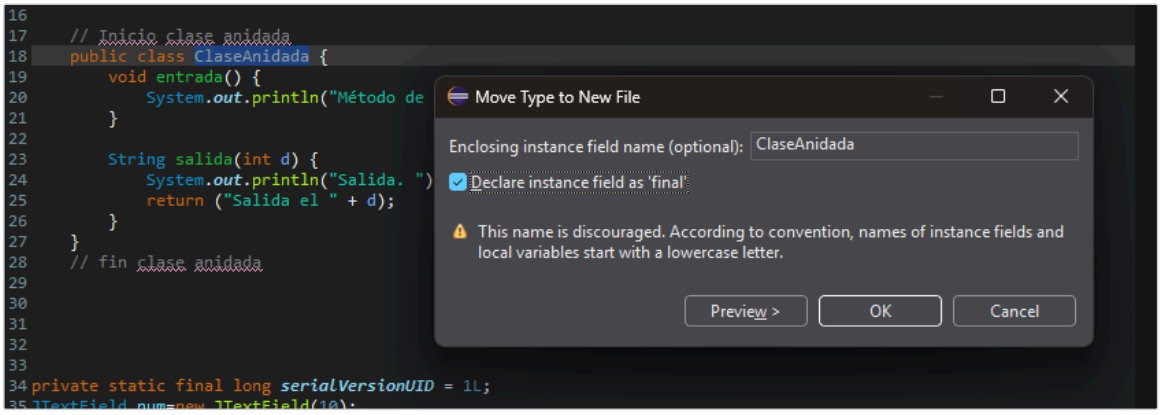
*Código en verporconsola*



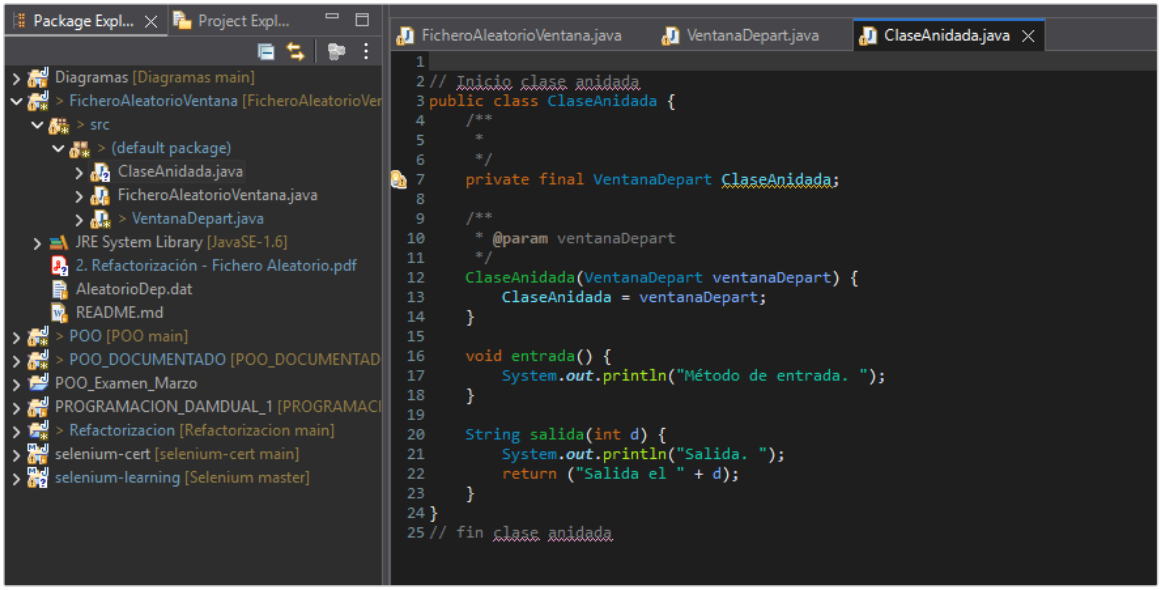

*"Método de entrada." visible en consola*

### b) Mover a fichero propio

*Refactor* → *Move Type to New File* → se crea `ClaseAnidada.java` automáticamente.



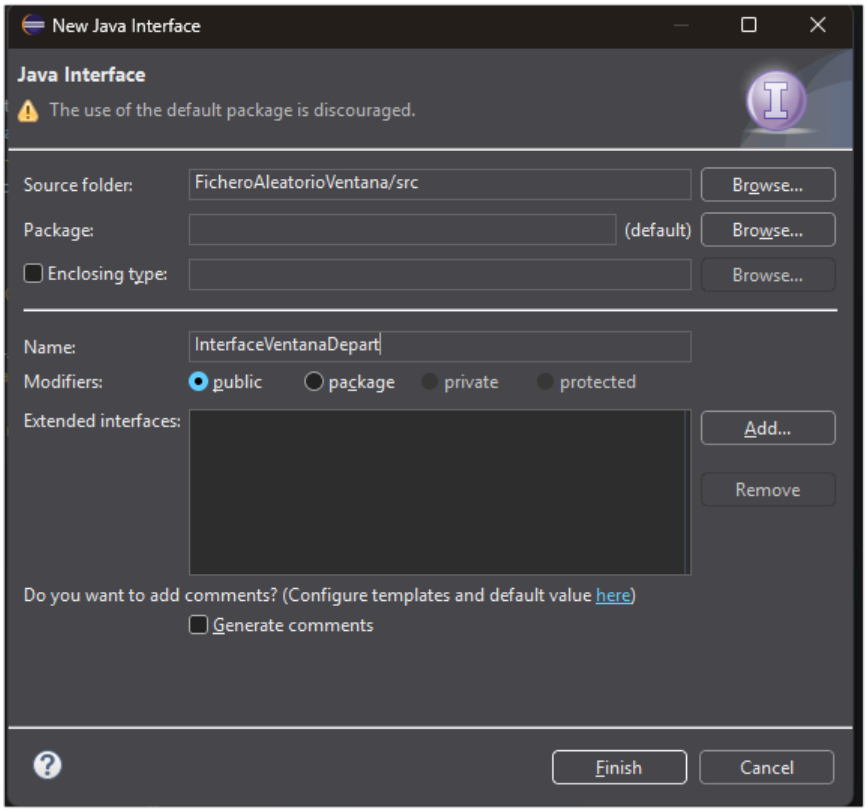

*Diálogo Move Type to New File*



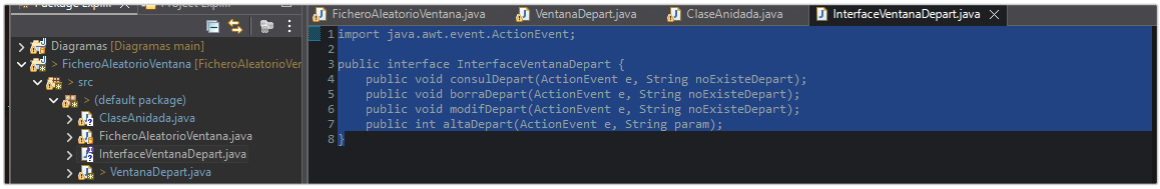

*ClaseAnidada.java como fichero independiente en Package Explorer*

## Apartado 9 — Interfaz `InterfaceVentanaDepart`

Se creó la interfaz con *New* → *Interface* con los cuatro métodos públicos del apartado 5. `VentanaDepart` modificada para implementarla.




*Diálogo New Java Interface con nombre InterfaceVentanaDepart*




*Código de InterfaceVentanaDepart.java con los 4 métodos declarados*

```
12 import javax.swing.*;
13
14 public class VentanaDepart extends JFrame implements ActionListener, InterfaceVentanaDepart   {
15
16
17 private static final long serialVersionUID = 1L;
18 JTextField num=new JTextField(10);
19 JTextField nombre=new JTextField(25);
20 JTextField loc=new JTextField(25);
21
22
23 JLabel mensaje=new JLabel(" ---------------------------- ");
24 JLabel titulo=new JLabel ("GESTIØN DE DEPARTAMENTOS.");
25
26 JLabel lnum = new JLabel ("NUMERO DEPARTAMENTO:");
27 JLabel lnom = new JLabel ("NOMBRE:");
28 JLabel lloc = new JLabel ("LOCALIDAD:");
29
30 JButton balta= new JButton("Insertar Depar.t");
```

*VentanaDepart implements ActionListener, InterfaceVentanaDepart*